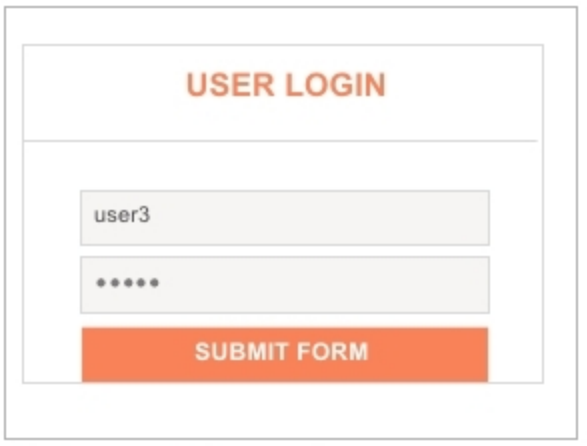

Figure 8: Page not accepting requests

than the set number of limited tries, which is three in this case, allowing the probability of finding the correct user name and corresponding password to decrease substantially. The same has been shown below and explained against localhost.

When a username in the database is entered with the corresponding password correctly, the output is a successful entry.

When the password entered is incorrect more than three times successively, the entry to the Web page is blocked. After three incorrect logins, it doesn't accept any more requests.

As seen in Figure 8, the page does not allow input of both the user name and password after three failed login attempts. In this way, brute forcing is detected and prevented.

When the command is tried again against localhost:

```
<wfuzz -z file>wordlist/others/common_
pass.txt -d "uname=FUZZ&pass=FUZZ" --hc
302 http://localhost/login.php
```

```
Total time: 0.096435
Processed Requests: 3
Filtered Requests: 0
Requests/sec.: 31.10901
```

Figure 9: Post requests processed -- 3

Here, the tool returns a *pycurl* error. Only three attempts were processed, while previously, anywhere from 52 to 946 attempts were processed. Brute forcing as seen earlier cannot be executed as all three attempts have been exhausted and have failed. Additionally, the tool takes only 0.09 seconds to stop processing, after which all three IDs/payloads are returned, as against the time taken above, which ranges from 16.16 to 48.13 seconds.

When fuzzing for paths/directories/files (common fuzzing), the command used is given below:

```
wfuzz -w wordlist/general/common.txt -
hc 404 http://testphp.vulnweb.com/
FUZZ
```

The number of requests returned is only three and this does not allow for obtaining the user name and password, thus ending in a failure. Similar to fuzzing for post requests, only three requests are processed and it shows a *pycurl* error.

In conclusion, fuzzing is prevented using a simple but robust method of limiting tries. This article focuses on the ways to use the said application in hacking as well as to prevent brute forcing. Wfuzz is a straightforward application that uses very simple command line inputs to hack Web applications, enabling developers to make their websites secure by finding the faults and vulnerabilities in their application. END

References

[1] 'Wfuzz the Web fuzzer': *https://github.com/xmendez/wfuzz*
[2] Wfuzz packet description: *https://tools.kali.org/web-applications/wfuzz*
[3] User guide: Basic usage, advanced usage and library guide: *https://wfuzz.readthedocs.io/en/latest/*
[4] More ways to prevent brute force attacks: *https://phoenixnap.com/kb/prevent-brute-force- attacks*

By: R. Dheekksha, Pravalikka Putha, B. Aparna and T. Subbulakshmi

R. Dheekksha, Pravalikka Putha and B. Aparna are FOSS enthusiasts.

Dr T. Subbulakshmi is a FOSS enthusiast and working as a professor, SCOPE at Vellore Institute of Technology, Chennai. Her research includes FOSS for security.